PERSONAL FINANCE MANAGER (PFM)

A PROJECT REPORT

SUBMITTED BY

[Student Name]
TU Registration No.: [Reg. No.]
Campus Roll No.: [Roll No.]

SUBMITTED TO

Department of Computer Science and Information Technology
[Campus/College Name]
Tribhuvan University

In partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science and Information Technology (B.Sc. CSIT)

[Month, Year]

TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY
[CAMPUS/COLLEGE NAME]
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project report prepared by **[Student Name]** entitled "**Personal Finance Manager (PFM)**" be accepted as a partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Information Technology. In my best knowledge, this is an original work in computer science.

.....

[Supervisor Name]

Supervisor

Department of Computer Science and Information Technology
Campus Name

Date

TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY
[CAMPUS/COLLEGE NAME]
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

LETTER OF APPROVAL

We certify that we have examined this project report entitled "**Personal Finance Manager (PFM)**" submitted by **[Student Name]** and reviewed the candidate's project work. We recommend that the project report be accepted as a partial fulfillment of the requirements for the degree of B.Sc. in Computer Science and Information Technology.

Evaluation Committee

.....	.....
[Supervisor Name]	**[Head of Department Name]**
Supervisor	Head of Department

.....
[External Examiner Name]
External Examiner

Date: .....

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, **[Supervisor Name]**, for their invaluable guidance, encouragement, and support throughout the course of this project. Their insights and feedback were instrumental in shaping the direction of this work.

I am also grateful to **[Head of Department Name]**, Head of the Department of Computer Science and Information Technology, for providing the necessary facilities and resources.

I would like to thank my friends and family for their unwavering support and motivation during the development of this project.

[Student Name]
Date

ABSTRACT

Managing personal finances is a crucial aspect of modern life, yet many individuals struggle with tracking expenses due to the tedious nature of manual entry and categorization. The **Personal Finance Manager (PFM)** is a hybrid mobile and web application designed to simplify expense tracking through the power of Natural Language Processing (NLP) and Artificial Intelligence.

The system allows users to log expenses using natural language phrases (e.g., "spent 500 on lunch"), which are then parsed and categorized automatically by Google's Gemini AI. Built using a robust tech stack comprising **React.js** for the frontend, **FastAPI (Python)** for the backend, **Supabase (PostgreSQL)** for the database, and **Capacitor** for mobile deployment, the application offers a seamless cross-platform experience. Key features include intelligent expense categorization, multi-group management for shared expenses, real-time financial analytics, and a voice-enabled interface. This project demonstrates the effective integration of modern web technologies and Generative AI to solve real-world financial management problems.

Keywords: *Personal Finance, Expense Tracking, Natural Language Processing, Artificial Intelligence, React, FastAPI, Gemini API.*

Contents

Acknowledgement **iv**

Abstract **v**

List of Abbreviations **xi**

1 INTRODUCTION **1**

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope and Limitation 2

1.4.1 Scope 2

1.4.2 Limitation 3

1.5 Development Methodology 3

1.6 Report Organization 3

2 BACKGROUND STUDY AND LITERATURE REVIEW **5**

2.1 Background Study 5

2.2 Literature Review 6

2.2.1 Mint (Intuit) 6

2.2.2 You Need A Budget (YNAB) 6

2.2.3 Splitwise 6

2.2.4 Research Gap 7

3 SYSTEM ANALYSIS AND DESIGN 8

3.1 System Analysis 8

3.1.1 Requirement Analysis 8

3.1.2 Feasibility Analysis 9

3.1.3 Data Modelling: ER Diagram 10

3.1.4 Process Modelling: DFD 11

3.1.5 Object Modelling: Object & Class Diagram 11

3.1.6 Dynamic Modelling: State & Sequence Diagram 12

3.1.7 Process Modelling: Activity Diagram 14

3.2 System Design 15

3.2.1 Architectural Design 15

3.2.2 Database Schema Design 15

3.2.3 Interface Design (UI/UX) 16

3.2.4 Refinement of Classes and Object 16

3.2.5 Component Diagram 16

3.2.6 Deployment Diagram 16

3.3 Algorithm Details 17

4 IMPLEMENTATION AND TESTING 18

4.1 Implementation 18

4.1.1 Tools Used 18

4.1.2 Implementation Details of Modules 18

4.2 Testing 20

4.2.1 Test Cases for Unit Testing 20

4.2.2 Test Cases for System Testing 20

5 CONCLUSION AND FUTURE RECOMMENDATIONS 21

5.1 Conclusion 21

5.2 Lesson Learnt/Outcome 21

5.3 Future Recommendations 22

List of Figures

3.1 Use Case Diagram 9

3.2 Entity Relationship (ER) Diagram 10

3.3 Data Flow Diagram (Level 0) 11

3.4 Class Diagram 12

3.5 State Diagram of Expense Processing 13

3.6 Sequence Diagram for Adding Expense 14

3.7 Activity Diagram 15

3.8 Component Diagram 16

3.9 Deployment Diagram 17

List of Tables

4.1 Unit Test Cases 20

4.2 System Test Cases 20

LIST OF ABBREVIATIONS

PFM	Personal Finance Manager
NLP	Natural Language Processing
AI	Artificial Intelligence
API	Application Programming Interface
SDLC	Software Development Life Cycle
UML	Unified Modeling Language
DFD	Data Flow Diagram
ER	Entity Relationship
UI/UX	User Interface / User Experience

Chapter 1

INTRODUCTION

1.1 Introduction

In an era of digital payments and complex financial obligations, effective management of personal finances has become a necessity. Individuals often struggle to keep track of their daily expenses, leading to poor financial health and lack of savings. Traditional methods of expense tracking, such as manual ledgers or spreadsheets, are time-consuming and prone to human error. Even mobile applications often require tedious manual entry of date, amount, category, and description for every transaction, leading to user drop-off.

The **Personal Finance Manager (PFM)** is designed to address these challenges by leveraging the power of Artificial Intelligence (AI) and Natural Language Processing (NLP). By allowing users to input expenses in natural language—as if they were texting a friend—the system removes the friction of manual data entry. Integrated with Google's Gemini API, the system intelligently understands the context of the expense, categorizes it automatically, and stores it for analysis. This project aims to democratize financial literacy and ease of management through a modern, user-friendly interface accessible on both web and mobile platforms.

1.2 Problem Statement

Despite the availability of numerous financial management tools, several key issues persist:

1. **High Data Entry Friction:** Users find it cumbersome to manually select categories, dates, and payment modes for every small transaction.
2. **Lack of Intelligent Categorization:** Most apps rely on rigid, pre-defined rules that

fail to understand context (e.g., "coffee with client" might be a business expense, not just "Food").

3. **Complex Interfaces:** Many financial apps are cluttered with complex charts and features that overwhelm the average user.

4. **Fragmented Experience:** Users often need separate apps for personal tracking and shared expenses (e.g., splitting bills with roommates).

1.3 Objectives

The primary objective of this project is to develop a smart, NLP-powered personal finance management system. The specific objectives are:

1. **To implement Natural Language Expense Logging:** Enable users to log expenses using simple text commands (e.g., "200 for taxi").

2. **To automate Expense Categorization:** Utilize Generative AI (Google Gemini) to accurately categorize transactions based on description and context.

3. **To provide Real-time Analytics:** Develop interactive dashboards to visualize spending habits, trends, and budget adherence.

4. **To support Multi-Group Management:** Facilitate shared expense tracking for families, roommates, or trip groups.

5. **To ensure Cross-Platform Accessibility:** Deploy the application as a Progressive Web App (PWA) and an Android application.

1.4 Scope and Limitation

1.4.1 Scope

The scope of the PFM project covers:

- **User Management:** Secure authentication and profile management via Supabase.

- **Expense Tracking:** Functionality to add, edit, and delete expenses using both text-based NLP input and manual forms.

- **AI Integration:** Integration with Google Gemini API for parsing unstructured text into structured JSON data.
- **Visualization:** Graphical representation of financial data using charts and graphs.
- **Platform:** The application is scoped for Web browsers (Chrome, Firefox, Safari) and Android devices (via Capacitor).

1.4.2 Limitation

- **Internet Dependency:** The AI features require an active internet connection to communicate with the Gemini API.
- **Language Support:** Currently, the NLP parser is optimized for English and Hinglish (Hindi-English mix) and may not support other regional languages effectively.
- **Bank Integration:** The system does not currently support automatic SMS reading or direct bank account syncing due to security and privacy constraints.
- **Scalability Constraints:** The free tier of Supabase and Gemini API imposes rate limits which may affect scalability under high load.

1.5 Development Methodology

The project follows the **Agile Software Development Methodology**. Agile was chosen for its iterative nature, allowing for continuous feedback and improvement. The development was divided into 2-week sprints:

- **Sprint 1:** Requirement gathering and UI protoyping.
- **Sprint 2:** Backend API development and Database setup.
- **Sprint 3:** Frontend integration and AI implementation.
- **Sprint 4:** Testing, Bug fixes, and Documentation.

1.6 Report Organization

This project report is organized into five chapters:

- **Chapter 1: Introduction** outlines the project background, problem, objectives, and scope.
- **Chapter 2: Background Study and Literature Review** discusses fundamental concepts and reviews similar existing systems.
- **Chapter 3: System Analysis and Design** details the requirement analysis, feasibility, and comprehensive system design using UML diagrams.
- **Chapter 4: Implementation and Testing** describes the tools used, implementation details, and testing strategies.
- **Chapter 5: Conclusion and Future Recommendations** summarizes the project outcomes and suggests future improvements.

Chapter 2

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

(Description of fundamental theories, general concepts and terminologies related to the project)

To understand the development of the Personal Finance Manager, it is essential to grasp the fundamental concepts and technologies involved:

- **Personal Finance Management (PFM):** The practice of budgeting, saving, and spending monetary resources over time, taking into account various financial risks and future life events.
- **Natural Language Processing (NLP):** A subfield of artificial intelligence describing the interaction between computers and human language. In this project, NLP is used to interpret unstructured expense descriptions (e.g., ”taxi 500”) into structured data (Category: Travel, Amount: 500).
- **Large Language Models (LLMs):** Deep learning algorithms that can recognize, summarize, translate, predict, and generate text and other forms of content based on knowledge gained from massive datasets. We utilize Google’s Gemini model for this purpose.
- **Client-Server Architecture:** A distributed application structure that partitions tasks or workloads between the providers of a resource or service, called servers, and service requesters, called clients. PFM uses React as the client and FastAPI as the server.
- **REST API (Representational State Transfer):** An architectural style for an application program interface (API) that uses HTTP requests to access and use data.

2.2 Literature Review

(Review of the similar projects, theories done by other researchers)

Several personal finance management applications exist in the market, each offering various features for expense tracking and budgeting. A critical review of these systems provides insights into current standards and gaps.

2.2.1 Mint (Intuit)

Mint was one of the most popular PFM tools, offering automatic bank synchronization, budgeting, and credit score tracking.

- **Overview:** Aggregates all financial accounts in one place.
- **Theory:** Automates tracking through direct bank connections.
- **Limitations:** Recently discontinued/migrated to Credit Karma, heavily ad-supported, privacy concerns with bank data.

2.2.2 You Need A Budget (YNAB)

YNAB focuses on zero-based budgeting, encouraging users to give every dollar a job.

- **Theory:** Proactive budgeting rather than reactive tracking.
- **Limitations:** Expensive subscription model, steep learning curve for new users, requires manual discipline.

2.2.3 Splitwise

Specifically designed for splitting bills and shared expenses.

- **Theory:** Simplified group debt simplification algorithms.
- **Limitations:** Not a full PFM (lacks personal budgeting, net worth tracking), limited analytics for individual spending.

2.2.4 Research Gap

Most existing research and products focus either on fully automated bank syncing (which raises privacy/security issues for local contexts) or fully manual entry (which causes user fatigue). There is a lack of ”Hybrid” systems that facilitate manual entry through intelligent automation like NLP commands. This project aims to fill this gap.

Chapter 3

SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

System analysis is a process of collecting and interpreting facts, identifying the problems, and decomposition of a system into its components.

3.1.1 Requirement Analysis

Requirement analysis focuses on the tasks that determine the needs or conditions to meet the new or altered product or project, taking account of the possibly conflicting requirements of the various stakeholders.

Functional Requirements

Functional requirements define the specific behavior or functions of the system.

1. **Authentication:** Users must be able to register and login securely.
2. **Expense Management:** Users can add, edit, and delete expenses using NLP text or manual forms.
3. **Categorization:** The system automatically categorizes expenses using AI.
4. **Visualization:** Users can view charts and graphs of their spending.
5. **Group Expenses:** Users can create groups and split bills.

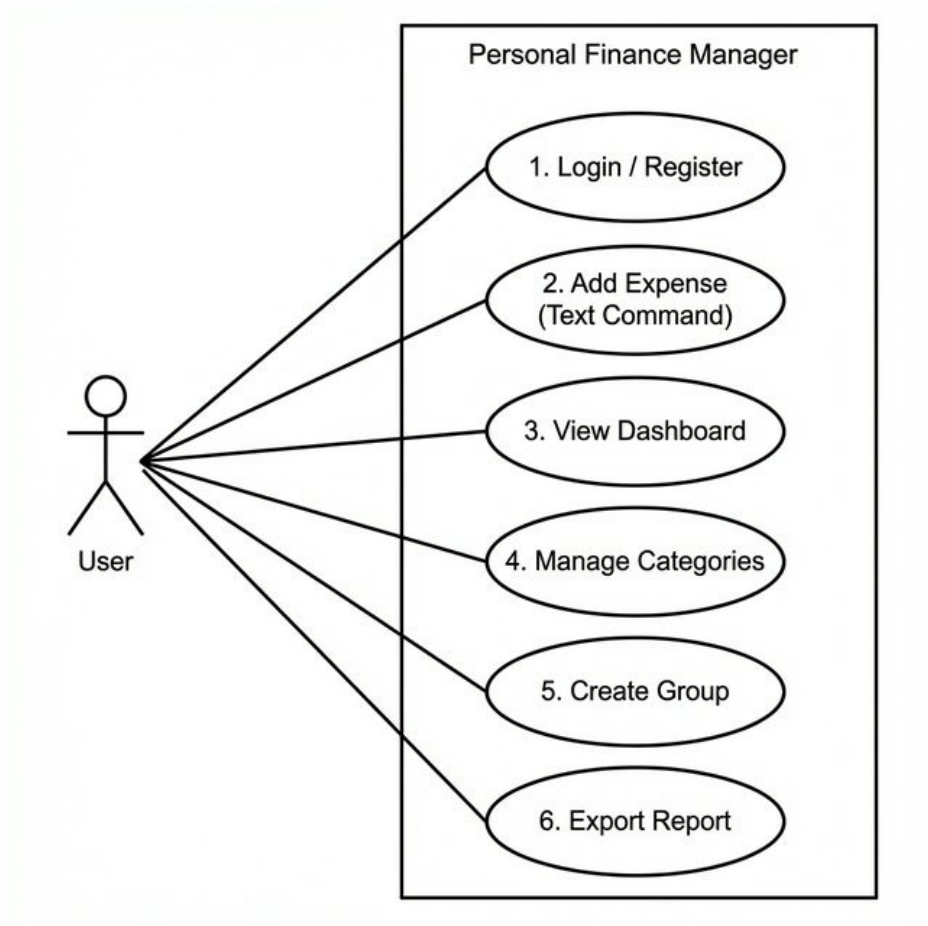

Figure 3.1: Use Case Diagram

Non-functional Requirements

1. **Performance:** Response time for NLP parsing should be under 2 seconds.
2. **Reliability:** The system should be available 99.9% of the time.
3. **Security:** Passwords must be hashed; API communication must be over HTTPS.
4. **Scalability:** System should handle concurrent users without degradation.

3.1.2 Feasibility Analysis

Technical Feasibility

The project uses React (Frontend), Python FastAPI (Backend), and PostgreSQL (Database), which are mature technologies. The integration with Gemini API is supported by official libraries. Thus, the project is technically feasible.

Operational Feasibility

The interface is designed to be intuitive, resembling a chat application. No special training is required for users, making it operationally feasible.

Economic Feasibility

The project relies on open-source technologies and free-tier cloud services (Supabase, Vercel, Google AI Studio). The primary investment is development time, making it economically feasible.

3.1.3 Data Modelling: ER Diagram

Entity-Relationship (ER) diagram shows the relationships of entity sets stored in a database.

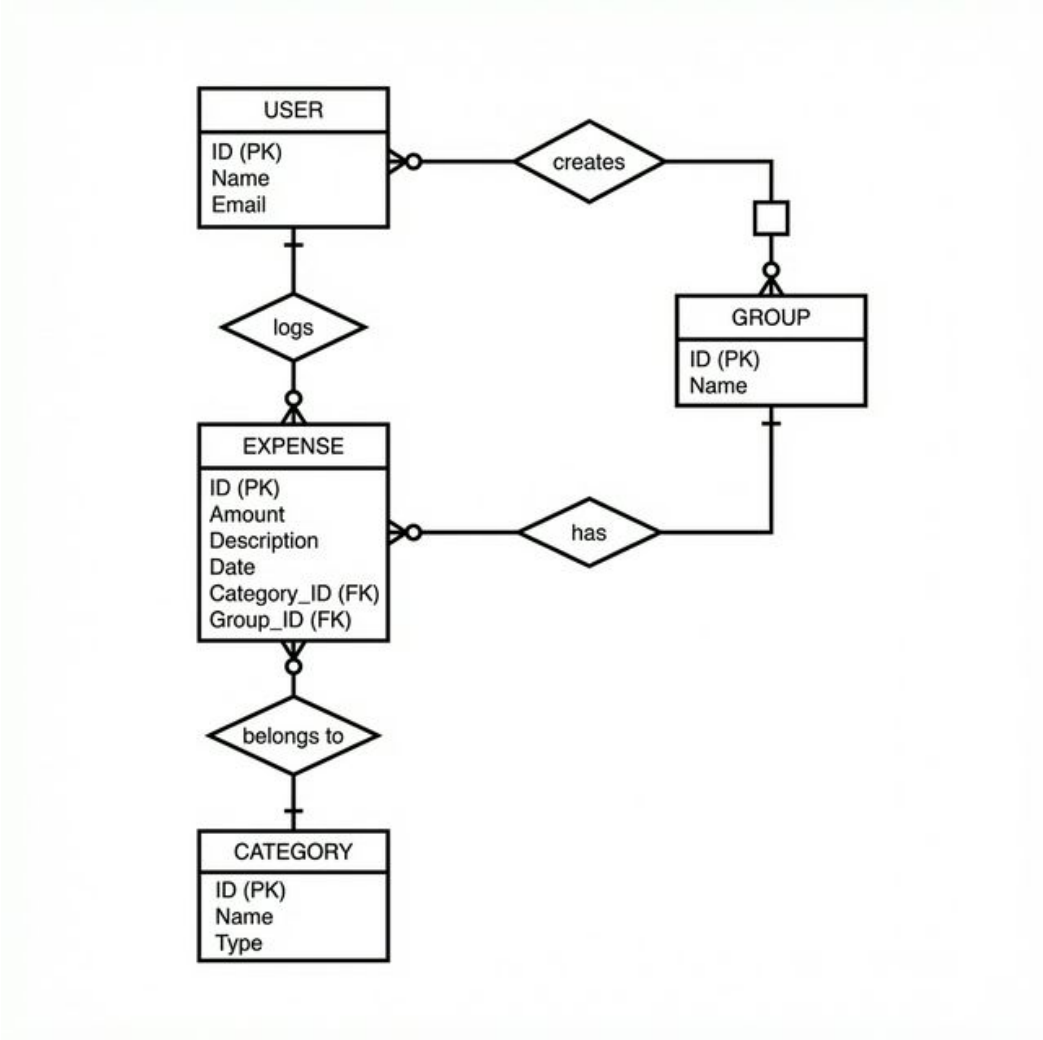

Figure 3.2: Entity Relationship (ER) Diagram

3.1.4 Process Modelling: DFD

Data Flow Diagram (DFD) maps the flow of information for any process or system.

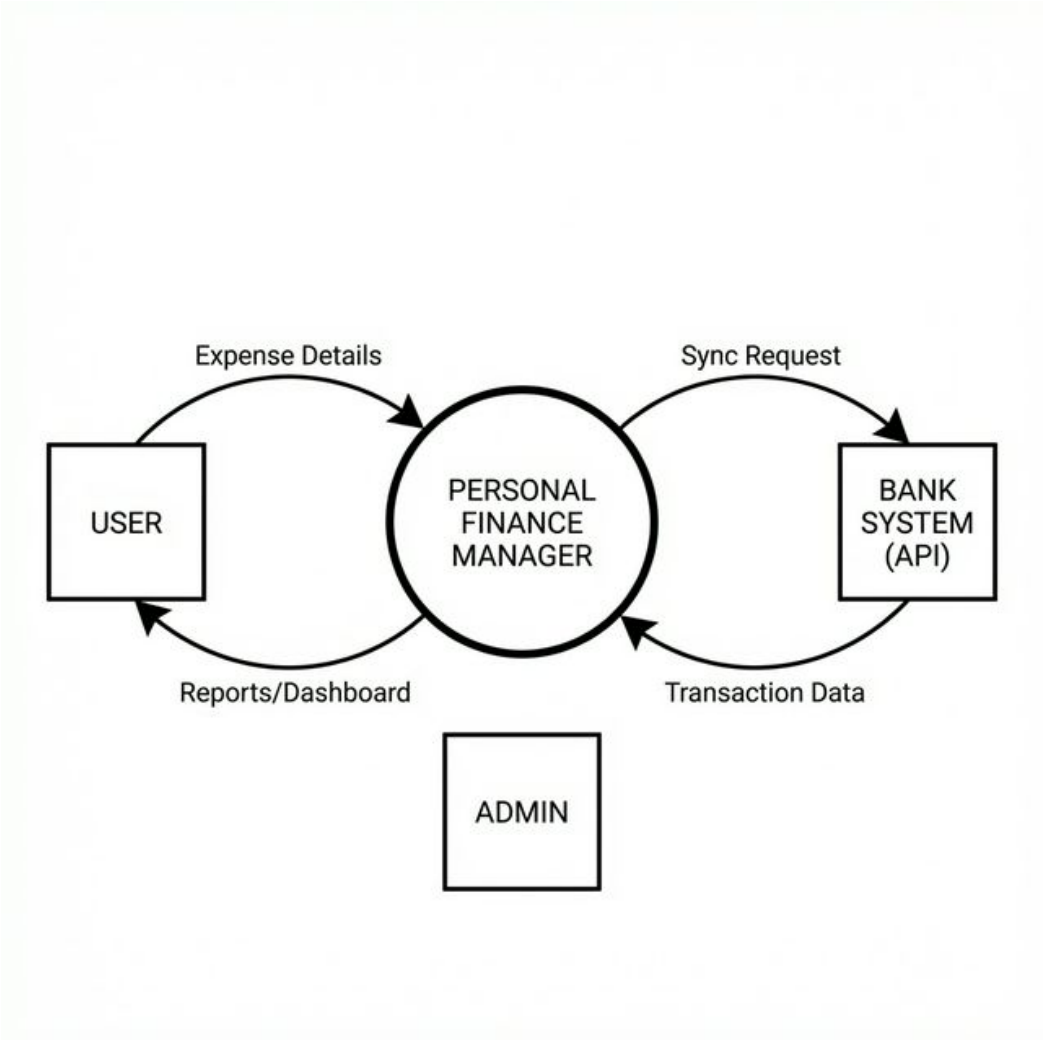

Figure 3.3: Data Flow Diagram (Level 0)

3.1.5 Object Modelling: Object & Class Diagram

Class diagram describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects.

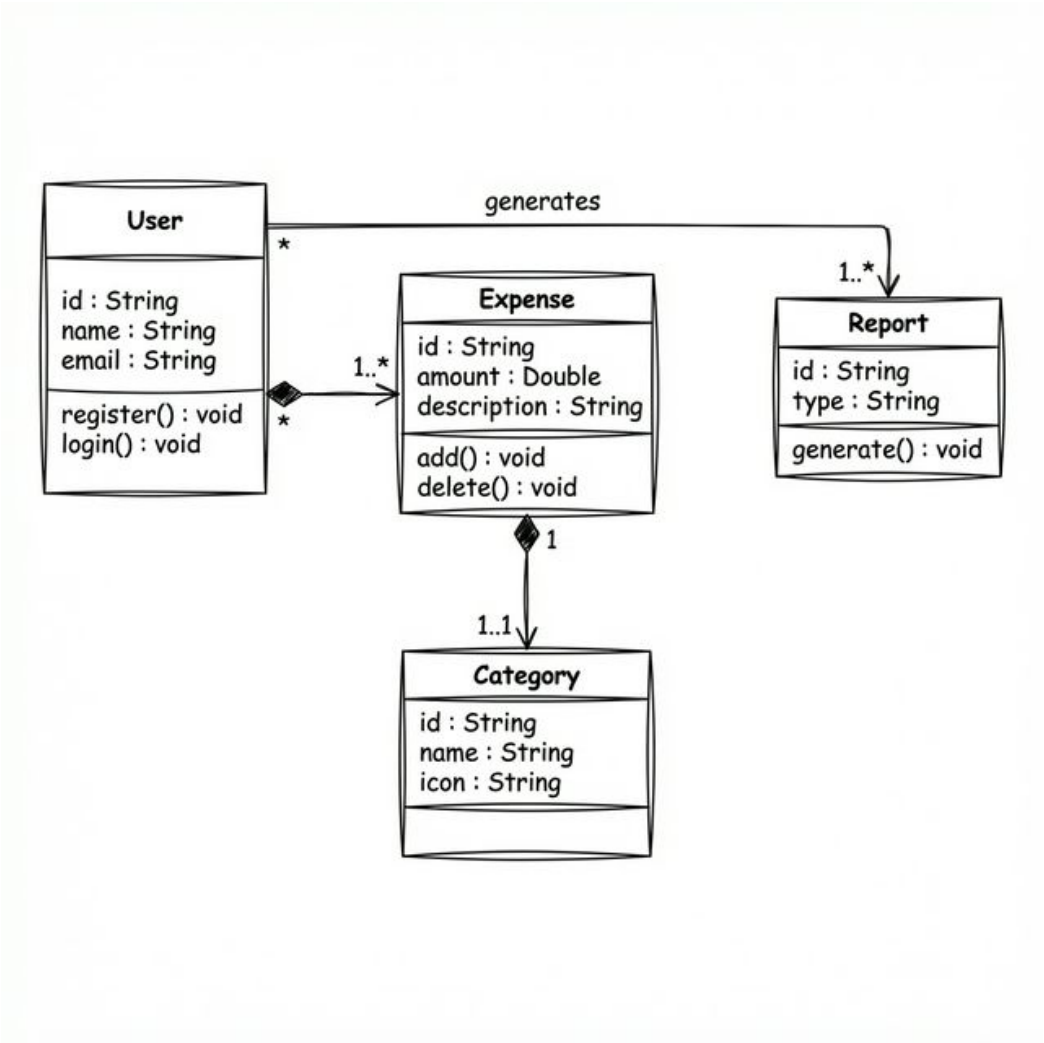

Figure 3.4: Class Diagram

3.1.6 Dynamic Modelling: State & Sequence Diagram

State Diagram

State diagram describes the behavior of a system.

images/state_diagram.png

Figure 3.5: State Diagram of Expense Processing

Sequence Diagram

Sequence diagram shows how objects operate with one another and in what order.

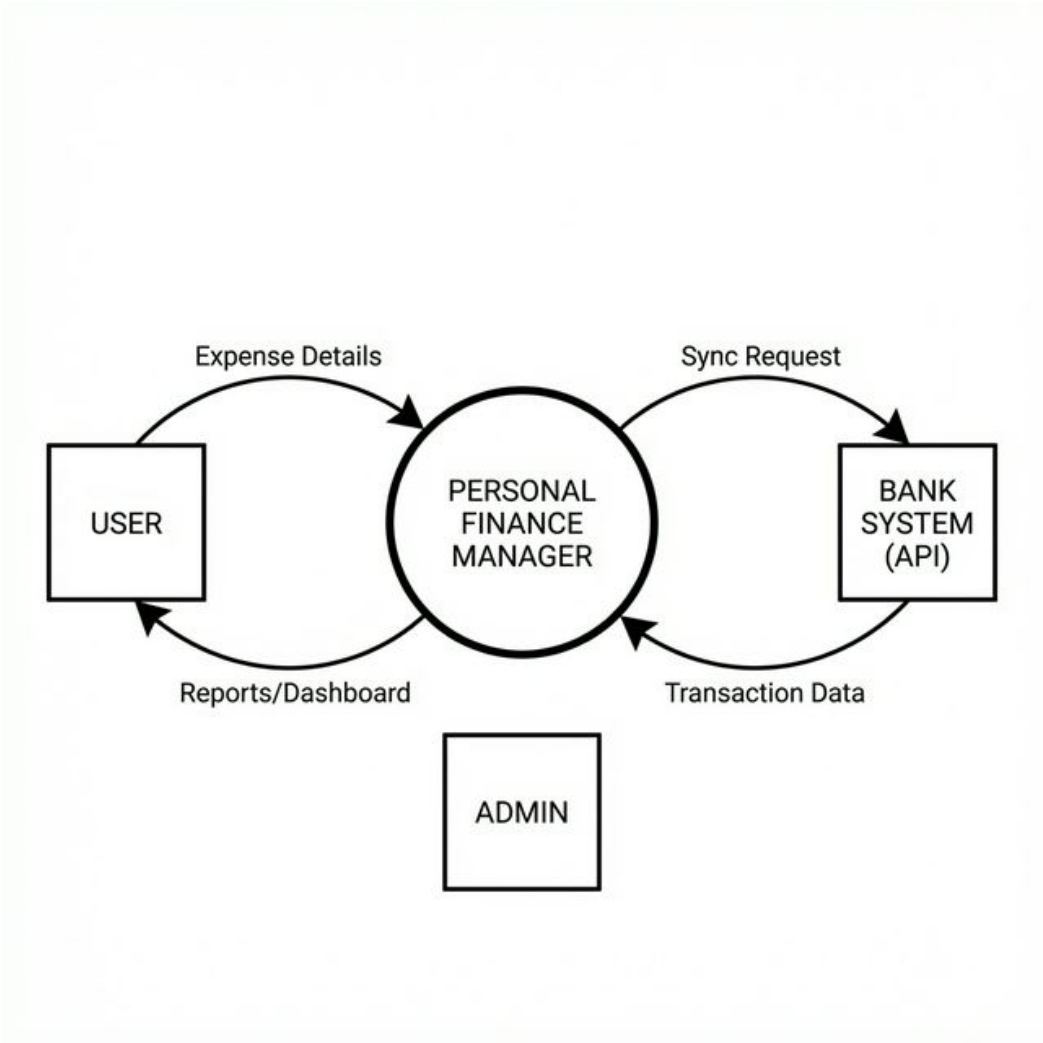

Figure 3.6: Sequence Diagram for Adding Expense

3.1.7 Process Modelling: Activity Diagram

Activity diagram graphically represents workflows of stepwise activities and actions with support for choice, iteration, and concurrency.

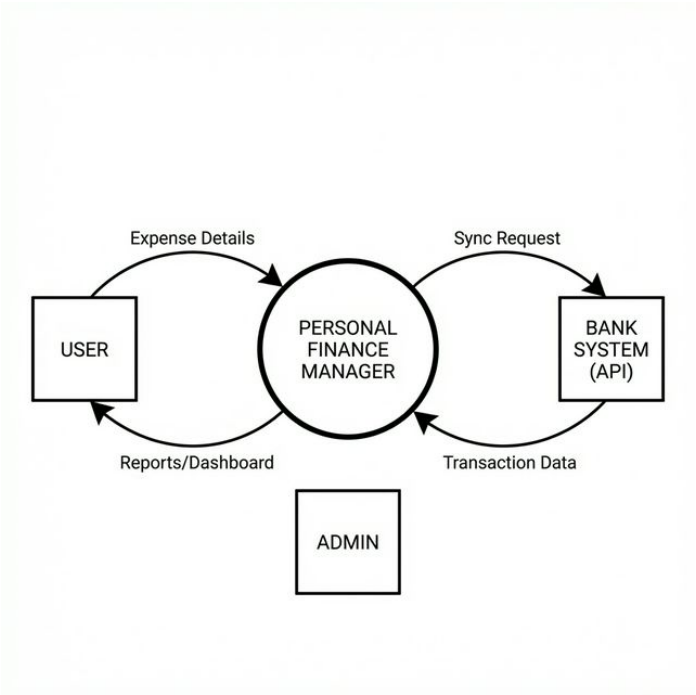

Figure 3.7: Activity Diagram

3.2 System Design

3.2.1 Architectural Design

The system uses a **Client-Server Architecture**. The client is a React Single Page Application (SPA) wrapped in Capacitor for mobile. The server is a REST API built with FastAPI.

3.2.2 Database Schema Design

The database is normalized to 3NF.

- **Users Table:** `id, restricted_uid, email, full_name, created_at`
- **Categories Table:** `id, name, type, icon`
- **Expenses Table:** `id, amount, description, date, user_id, category_id, group_id`
- **Groups Table:** `id, name, created_by`

3.2.3 Interface Design (UI/UX)

The UI is designed for mobile-first usage. It features a bottom navigation bar, a central floating action button for adding expenses, and a card-based layout for listing transactions.

3.2.4 Refinement of Classes and Object

The initial class model was refined to include helper classes for API communication (`ApiService`) and state management (`ExpenseContext`).

3.2.5 Component Diagram

Component diagram depicts how components are wired together to form larger components or software systems.

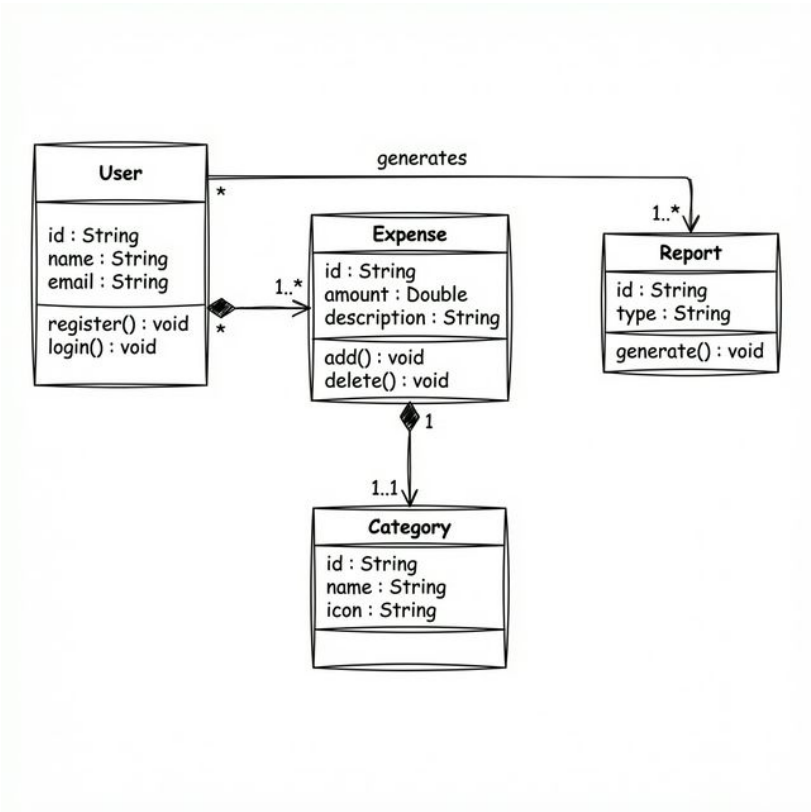

Figure 3.8: Component Diagram

3.2.6 Deployment Diagram

Deployment diagram shows the physical hardware on which the software components run.

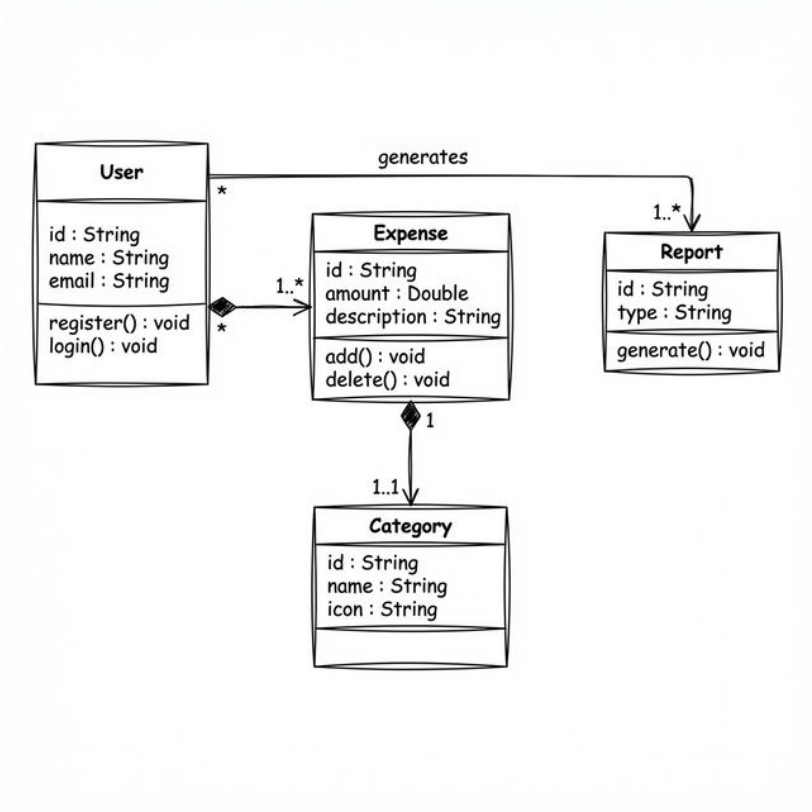

Figure 3.9: Deployment Diagram

3.3 Algorithm Details

The NLP processing algorithm is central to the system:

1. **Input:** Raw text string S from user.
2. **Tokenization:** Send S to LLM with schema instruction.
3. **Extraction:** LLM identifies entities: *Amount*, *Currency*, *Category* (mapped to enum), *Date* (relative to absolute).
4. **Validation:** Backend defines Pydantic models to validate the JSON structure.
5. **Output:** Structured ‘ExpenseCreate‘ object or Error.

Chapter 4

IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools Used

The project was implemented using the following set of tools and technologies:

- **Frontend:** React.js (v18), TailwindCSS (Styling), Recharts (Data Visualization), Capacitor (Mobile Build).
- **Backend:** Python 3.10+, FastAPI (Web Framework), Uvicorn (ASGI Server), Google Gemini SDK (AI).
- **Database Platform:** Supabase (PostgreSQL), Supabase Auth.
- **CASE Tools:** Visual Studio Code, Git, GitHub.

4.1.2 Implementation Details of Modules

User Authentication Module

Implemented using Supabase Auth. Provides functions for `signUp()`, `signInWithPassword()`, and `signOut()`. Handles session management and JWT token verification.

Expense Parsing Module (Backend)

The core feature of the application is the NLP parser. Below is the Python implementation using FastAPI and Gemini:

```python
# backend/api/expenses.py (Snippet)

@router.post("/parse")
async def parse_expense(text: str):
    """
    Parses natural language text into structured expense data.
    """
    prompt = f"""
    Extract the following details from the text: "{text}"
    - Amount (number)
    - Category (Enum: Food, Travel, Bills, Entertainment, Health, Other)
    - Description (string)

    Return JSON format only.
    """

    response = model.generate_content(prompt)
    try:
        data = json.loads(response.text)
        return {"status": "success", "data": data}
    except json.JSONDecodeError:
        return {"status": "error", "message": "Failed to parse"}
```

Listing 4.1: Backend API for Expense Parsing

Expense Management Module (Frontend)

The frontend manages state using React Context API to ensure data consistency across components.

```javascript
// src/context/ExpenseContext.js (Snippet)

export const ExpenseProvider = ({ children }) => {
    const [expenses, setExpenses] = useState([]);

    const addExpense = async (text) => {
        const parsed = await api.parseExpense(text);
        if (parsed.status === "success") {
            const newExpense = await api.saveExpense(parsed.data);
            setExpenses([newExpense, ...expenses]);
        }
    };

    return (
        <ExpenseContext.Provider value={{ expenses, addExpense }}>
```

```
            {children}
        </ExpenseContext.Provider>
    );
};
```

Listing 4.2: React Context for Expenses

4.2 Testing

4.2.1 Test Cases for Unit Testing

Unit tests were written for individual backend functions, particularly the NLP parsing logic.

Test ID	Input	Expected Output	Result
UT-01	”Food 500”	{amount: 500, category: ”Food”}	Pass
UT-02	”Taxi 200”	{amount: 200, category: ”Travel”}	Pass
UT-03	”Invalid”	Error Message	Pass

Table 4.1: Unit Test Cases

4.2.2 Test Cases for System Testing

System tests verified the end-to-end flow.

Test ID	Scenario	Expected Outcome	Result
ST-01	User Logs In	Dashboard loads with user data	Pass
ST-02	Add Expense via NLP	Expense appears in list	Pass
ST-03	View Reports	Charts render with correct data	Pass
ST-04	Mobile View	Layout adapts to screen size	Pass

Table 4.2: System Test Cases

Chapter 5

CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

The **Personal Finance Manager (PFM)** project successfully demonstrates the potential of integrating Generative AI into everyday utility applications. By simplifying the expense logging process through Natural Language Processing, the system addresses the primary pain point of traditional finance apps—manual data entry friction. The use of a modern tech stack (React, FastAPI, Supabase) ensures the application is scalable, responsive, and maintainable.

The project met functionality objectives, including accurate parsing of expense text using Google Gemini, real-time data visualization, and secure user authentication. This system provides a robust foundation for personal financial management and showcases the practical application of Large Language Models (LLMs) in software engineering.

5.2 Lesson Learnt/Outcome

Through the development of this project, several key lessons were learned:

- **AI Integration:** Learned how to effectively prompt engineer for consistent JSON outputs from LLMs.
- **State Management:** Gained deep understanding of React Context API for managing cross-component state.
- **Mobile Development:** Understood the nuances of building cross-platform apps using Capacitor.
- **API Design:** Learned importance of robust error handling in REST APIs.

5.3 Future Recommendations

While the current system is functional, several enhancements can be made in future iterations:

1. **Bank SMS Integration:** Automate expense logging by reading transaction SMS messages (Android only).
2. **OCR for Receipts:** Implement Optical Character Recognition (OCR) to allow users to scan physical receipts.
3. **Budget Alerts:** Add push notifications when users exceed their set category budgets.
4. **Investment Tracking:** Extend the system to track investments (Stocks, Mutual Funds) alongside daily expenses.
5. **Offline Mode:** Enhance the PWA capabilities to allow offline logging, syncing when the internet is restored.

References

[1] Tiangolo. (n.d.). *FastAPI Framework*. [Online]. Available: https://fastapi.tiangolo.com/

[2] Meta Platforms. (n.d.). *React - A JavaScript library for building user interfaces*. [Online]. Available: https://reactjs.org/

[3] Google. (2024). *Gemini API Overview*. [Online]. Available: https://ai.google.dev/

[4] Supabase Inc. (n.d.). *The Open Source Firebase Alternative*. [Online]. Available: https://supabase.com/

[5] I. Sommerville, *Software Engineering*, 9th ed. Boston, MA: Addison-Wesley, 2011.

[6] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY: McGraw-Hill Education, 2014.